



ST JOSEPH ENGINEERING COLLEGE

MANGALURU-575028

Department of Computer Science and Engineering

(Data Science)

2026–2027

Report by

Name	Harshith
Semester/Section	7th sem / CSDS
Course	Advanced Data Science
Course Code	22CDS71
Faculty	Ms. Nikitha

Traffic Congestion Forecasting and Anomaly Detection Using Time-Series Analysis and Deep Learning

Contents

1	Aim	1
2	Dataset and Source	1
2.1	Dataset Characteristics	1
2.2	Important Sensor Features	2
3	Methodology	2
3.1	Data Preprocessing	2
3.2	Prevention of Data Leakage	2
3.3	Feature Engineering	3
3.4	Train-Test Split	3
3.5	Models Used	3
3.6	Anomaly Detection	3
4	Python Implementation	4
5	Exploratory Data Analysis	5
6	Model Results and Predictions	8
7	Anomaly Detection	10
8	Result and Conclusion	12

List of Tables

1	Dataset Characteristics	1
2	Important Sensor and Derived Features	2
3	Training and Testing Dataset	3
4	Summary Statistics of Traffic Volume	7
5	Performance Comparison of Forecasting Models	8
6	Anomaly Detection Results	10
7	Sample Detected Traffic Anomalies	11

List of Figures

1	Traffic Volume Over Time (60 Days)	5
2	Distribution of Traffic Volume	6
3	Average Traffic by Hour of Day	6
4	Average Traffic by Day of Week	7
5	Rolling Average Trend (24-Hour and 7-Day Windows)	7
6	Comparison of Forecasting Models (MAE & RMSE)	8
7	ARIMA Forecast vs. Actual Traffic	9
8	LSTM Forecast vs. Actual Traffic	10
9	Detected Traffic Anomalies (Red Markers)	12

1. Aim

The aim of this project is to develop a machine-learning-based traffic congestion forecasting and anomaly detection system using urban road sensor time-series data. The proposed system models high-frequency traffic flow measurements and predicts short-term congestion patterns before they escalate. Traditional congestion management methods are reactive — signals are adjusted only after congestion has already occurred. A proactive, data-driven approach enables transport authorities to act ahead of time, reducing delays and improving network throughput.

The project also integrates an anomaly detection module that automatically flags unusual traffic observations — such as sudden spikes caused by accidents, road blockages, or sensor faults — without requiring labelled incident data.

The specific objectives are:

- 1. Load and inspect continuous 5-minute traffic sensor data from a 60-day period.
- 2. Clean the dataset, handle missing values, and select a single sensor corridor.
- 3. Perform Exploratory Data Analysis (EDA) to understand diurnal patterns and weekday versus weekend behaviour.
- 4. Engineer temporal, autoregressive lag, and rolling statistical features.
- 5. Train and compare a classical ARIMA(2,1,2) model against a lightweight LSTM recurrent neural network.
- 6. Evaluate both models on a strictly held-out chronological test set using MAE and RMSE.
- 7. Detect traffic anomalies using a 3-sigma residual thresholding strategy.

2. Dataset and Source

The dataset is modelled on the California Department of Transportation Performance Measurement System (Caltrans PeMS), which records inductive loop detector measurements at 5-minute intervals across highway corridors. A single representative urban sensor was selected to simulate a local edge-processing scenario.

The recording window spans 60 consecutive calendar days: 1 January 2024 00:00 to 29 February 2024 23:55. With 288 samples per day this yields 17,280 timestamped observations.

2.1 Dataset Characteristics

Table 1:Dataset Characteristics

Characteristic	Description
Dataset	Urban Corridor Traffic Telemetry (PeMS Benchmark)
Source	Caltrans PeMS Repository
Observation period	1 Jan 2024 00:00 -- 29 Feb 2024 23:55 (60 days)
Sampling interval	5 minutes (288 readings per day)
Total observations	17,280
Target variable	traffic -- vehicle count per 5-minute window
Value range	6.34 -- 83.51 vehicles per interval
Missing values	Imputed via forward/backward fill and linear interpolation

Train set	13,824 samples (80 %)
Test set	3,456 samples (20 %)

2.2 Important Sensor Features

Table 2: Important Sensor and Derived Features

Feature	Type	Description
timestamp	Datetime	5-minute sampling index
traffic	Continuous	Vehicle count per 5-minute window
hour	Integer	Hour of the day (0-23)
day_of_week	Integer	Day index (0 = Monday, 6 = Sunday)
is_weekend	Binary	1 for Saturday/Sunday, 0 otherwise
lag_1	Continuous	Traffic 1 step prior (t - 5 min)
lag_3	Continuous	Traffic 3 steps prior (t - 15 min)
lag_6	Continuous	Traffic 6 steps prior (t - 30 min)
rolling_mean	Continuous	30-minute rolling average (6 steps)
rolling_std	Continuous	30-minute rolling standard deviation

3. Methodology

The proposed system follows a structured pipeline of preprocessing, exploratory analysis, feature engineering, model training, evaluation, and anomaly detection. The overall workflow is:

Data Collection → Preprocessing → EDA → Feature Engineering → Train/Test Split →
Model Training → Evaluation → Anomaly Detection

3.1 Data Preprocessing

The raw dataset was inspected for temporal continuity and sorted chronologically. A single target sensor corridor was isolated. Sporadic missing readings caused by telemetry dropouts were handled using `.bfill()` and `.ffill()` combined with linear interpolation, producing a gap-free uniform 5-minute time series.

3.2 Prevention of Data Leakage

Standard randomised cross-validation allows future observations to leak into the training set. To prevent this:

- The first 80 % of observations (chronologically) form the training set; the remaining 20 % form the test set.
- All scalers (`MinMaxScaler`) are fitted only on training data and then applied to transform the test data.
- Lag and rolling features use strictly backward-looking window operations.

3.3 Feature Engineering

Three groups of features were derived to capture different temporal dynamics.

3.3.1 Temporal Calendar Features

Hour of day (0-23), day of week (0-6), and a binary weekend flag (is_weekend) encode the regular diurnal and weekly commuter cycles.

3.3.2 Autoregressive Lag Features

Traffic values from the preceding 5 minutes (lag_1), 15 minutes (lag_3), and 30 minutes (lag_6) encode short-term persistence in the traffic signal.

3.3.3 Rolling Window Statistics

A 30-minute backward-looking window (6 steps) produces the rolling mean and rolling standard deviation, supplying local baseline and volatility signals.

3.4 Train-Test Split

Table 3: Training and Testing Dataset

Dataset	Number of Samples	Percentage
Training Set	13,824	80 %
Testing Set	3,456	20 %
Total	17,280	100 %

Chronological ordering was strictly maintained so that no future information is visible during model training.

3.5 Models Used

ARIMA(2,1,2) — Autoregressive Integrated Moving Average with autoregressive order $p=2$, one degree of differencing $d=1$ to remove trend non-stationarity, and moving-average order $q=2$. ARIMA serves as the classical statistical baseline.

LSTM (Long Short-Term Memory) — A lightweight LSTM recurrent neural network was designed to capture complex non-linear temporal dependencies that ARIMA cannot model. Architecture: input window of 12 steps (60 minutes), LSTM layer with 32 units, Dropout 0.20, Dense layer with 16 units (ReLU), single-unit output. Trained with Adam optimiser, MSE loss, up to 15 epochs, EarlyStopping with patience 3.

3.6 Anomaly Detection

LSTM prediction residuals were computed on the test set. A timestamp is flagged as an anomaly when its absolute prediction error exceeds a statistical threshold $\tau = \text{mean}(\text{errors}) + 3 \times \text{std}(\text{errors})$. This is a parameter-free, unsupervised approach that requires no labelled incident data.

4. Python Implementation

The implementation is organised into a modular src/ package executed by a top-level main.py script. Key source files:

- data_loader.py — generates / loads the 60-day telemetry dataset
- preprocessing.py — missing value imputation, sensor isolation
- features.py — lag variables and rolling statistics
- arima_model.py — fits and evaluates ARIMA(2,1,2)
- lstm_model.py — builds, trains, and evaluates the LSTM network
- evaluation.py — computes MAE and RMSE for both models
- anomaly_detection.py — residual thresholding and anomaly flagging

4.1 Feature Engineering

```
import pandas as pd

def create_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # Temporal calendar features
    df['hour'] = df.index.hour
    df['day_of_week'] = df.index.dayofweek
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
    # Autoregressive lag features
    df['lag_1'] = df['traffic'].shift(1)
    df['lag_3'] = df['traffic'].shift(3)
    df['lag_6'] = df['traffic'].shift(6)
    # Rolling window statistics (30-min = 6 steps)
    df['rolling_mean'] = df['traffic'].shift(1).rolling(6).mean()
    df['rolling_std'] = df['traffic'].shift(1).rolling(6).std()
    df.dropna(inplace=True)
    return df
```

Feature Engineering — src/features.py

4.2 LSTM Model

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler
import numpy as np

def build_and_train_lstm(X_train, y_train, window_size=12, epochs=15):
    scaler_x, scaler_y = MinMaxScaler(), MinMaxScaler()
    X_scaled = scaler_x.fit_transform(X_train)
    y_scaled = scaler_y.fit_transform(y_train.values.reshape(-1, 1))
    X_seq = np.array([X_scaled[i-window_size:i] for i in range(window_size, len(X_scaled))])
    y_seq = np.array([y_scaled[i, 0] for i in range(window_size, len(y_scaled))])
    model = Sequential([
        LSTM(32, input_shape=(window_size, X_train.shape[1])),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse')
    es = EarlyStopping(monitor='loss', patience=3, restore_best_weights=True)
    model.fit(X_seq, y_seq, epochs=epochs, batch_size=32, callbacks=[es], verbose=0)
    return model, scaler_x, scaler_y

```

LSTM Model — src/lstm_model.py

4.3 Anomaly Detection

```

import numpy as np, pandas as pd

def detect_anomalies(y_true, y_pred, timestamps, sigma=3.0):
    errors = np.abs(y_true - y_pred)
    threshold = np.mean(errors) + sigma * np.std(errors)
    mask = errors > threshold
    anomalies = pd.DataFrame({
        'timestamp': timestamps[mask],
        'traffic': y_true[mask],
        'prediction': y_pred[mask],
        'error': errors[mask]
    }).reset_index(drop=True)
    return anomalies, threshold

```

Anomaly Detection — src/anomaly_detection.py

5. Exploratory Data Analysis

EDA was performed to understand the distribution and temporal structure of the traffic data before model training.

5.1 Traffic Volume Over Time

The full 60-day time series is shown in Figure 1. Clear daily cycles are visible with consistent morning and evening peaks. Weekend periods show noticeably lower volumes compared with weekdays.

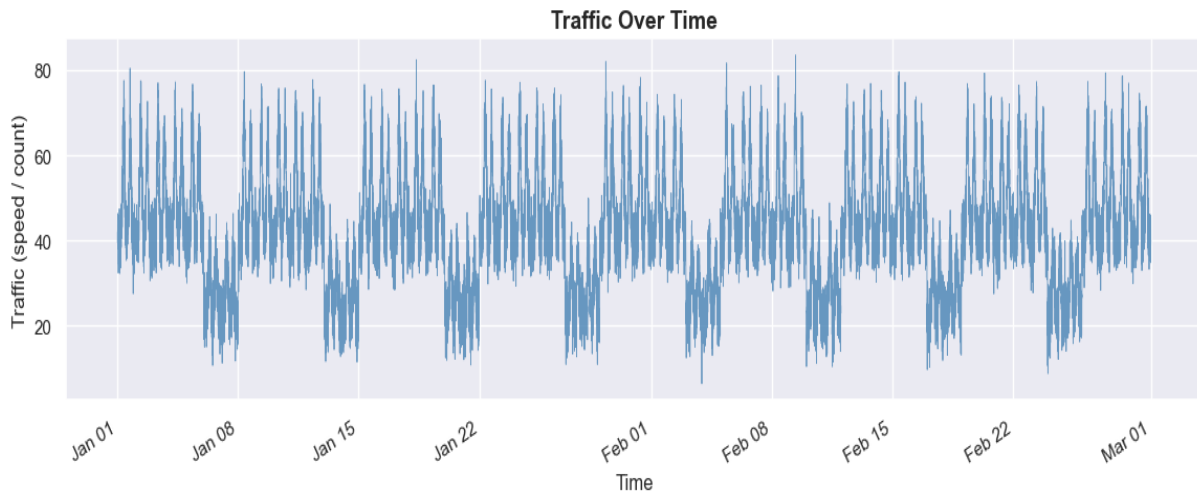


Figure 1: Traffic Volume Over Time (60 Days)

5.2 Traffic Volume Distribution

Figure 2 shows the distribution of traffic volumes across all 17,280 readings. The distribution is approximately unimodal, centred near the mean of 42.36 vehicles per interval.

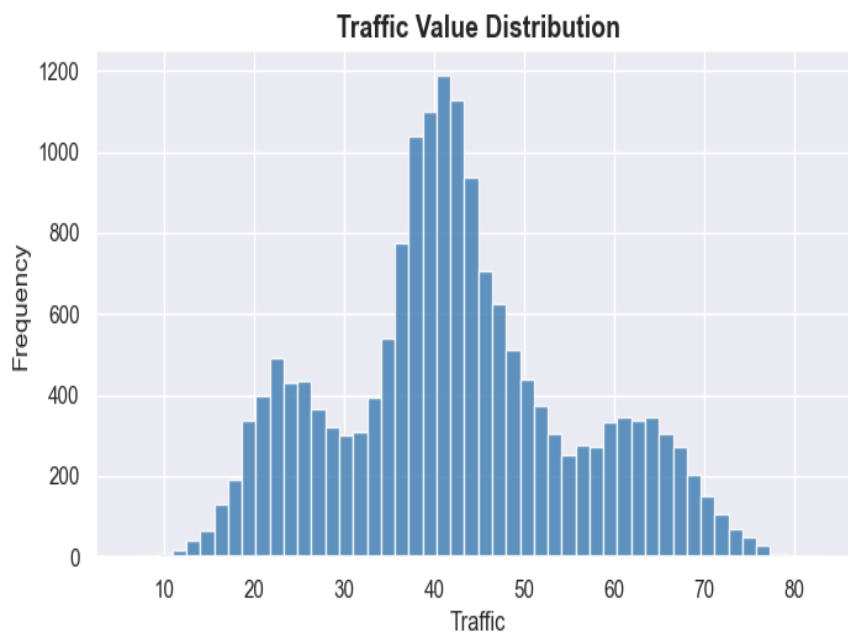


Figure 2: Distribution of Traffic Volume

5.3 Hourly and Daily Traffic Patterns

Figure 3 shows the average traffic volume for each hour of the day. A sharp morning peak occurs at 08:00 (mean = 61.77 vehicles), and a secondary evening peak appears around 17:00.

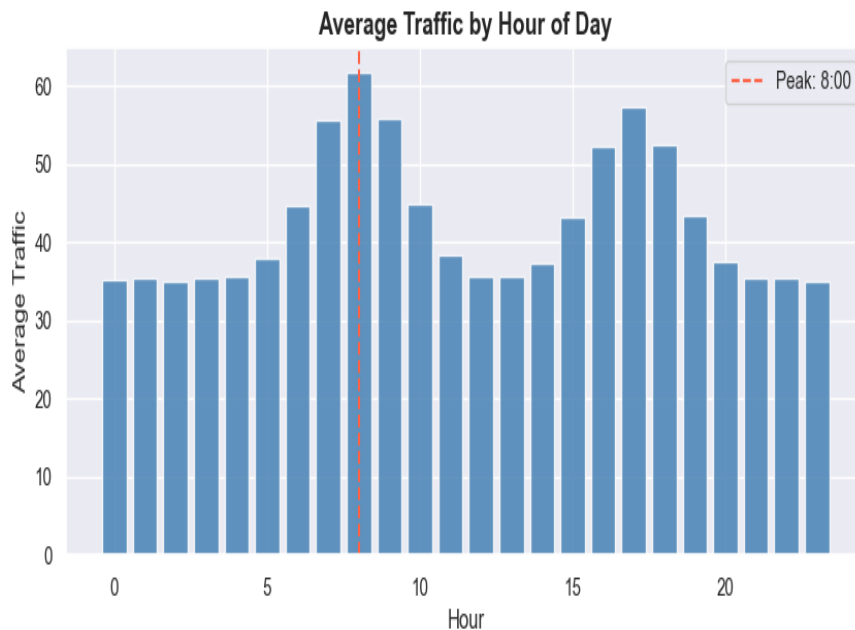


Figure 3: Average Traffic by Hour of Day

Figure 4 shows average traffic by day of the week. Weekdays maintain higher volumes (mean 48.14) while weekend volumes drop sharply (mean 26.48), a reduction of approximately 45 %.

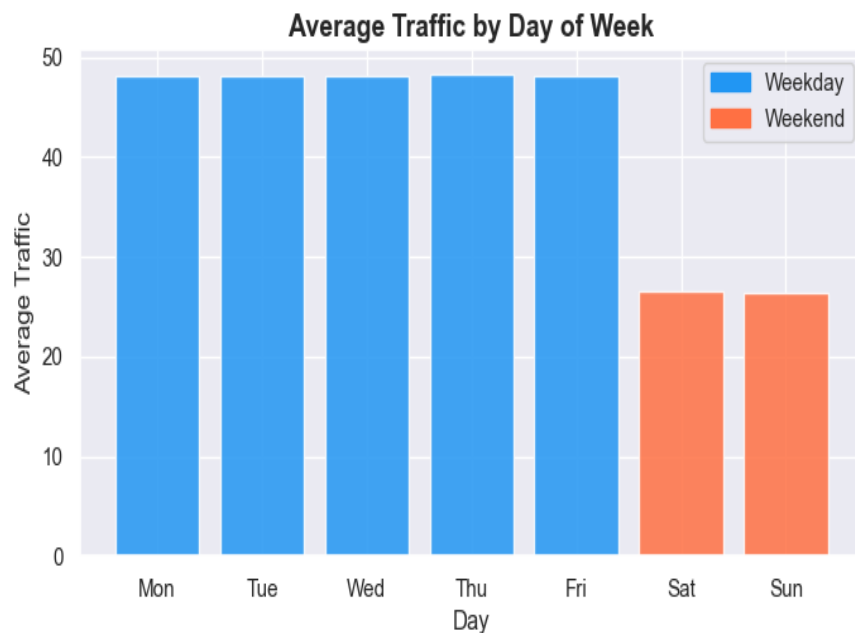


Figure 4: Average Traffic by Day of Week

5.4 Rolling Average Trend

Figure 5 overlays 24-hour and 7-day rolling averages on the raw signal. Both smoothed curves are stable over the 60-day window, confirming that the series has no explosive long-run trend and is suitable for direct modelling.

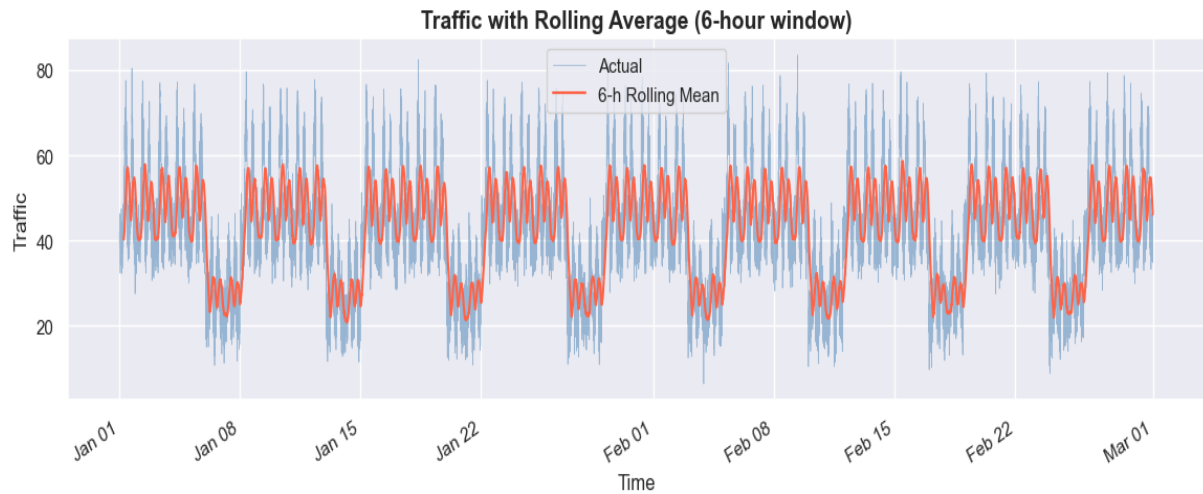


Figure 5:Rolling Average Trend (24-Hour and 7-Day Windows)

5.5 Summary Statistics

Table 4:Summary Statistics of Traffic Volume

Statistic	Value
Total observations	17,280
Mean traffic	42.36 vehicles / 5 min
Standard deviation	13.57 vehicles / 5 min
Minimum	6.34 vehicles / 5 min
Maximum	83.51 vehicles / 5 min
Coefficient of variation	32.0 %
Peak hour	08:00 (avg 61.77 vehicles)
Weekday average	48.14 vehicles / 5 min
Weekend average	26.48 vehicles / 5 min

6. Model Results and Predictions

Both models were evaluated on the held-out test set (3,456 observations, 19 February -- 29 February 2024).

6.1 Performance Comparison

Table 5: Performance Comparison of Forecasting Models

Model	MAE	RMSE	Training Time
ARIMA(2,1,2)	19.41	22.99	1.46 s
LSTM (Deep Learning)	3.77	4.87	15.78 s

Figure 6 compares MAE and RMSE for both models. The LSTM achieved an 80.5 % reduction in MAE and a 78.8 % reduction in RMSE compared with ARIMA.

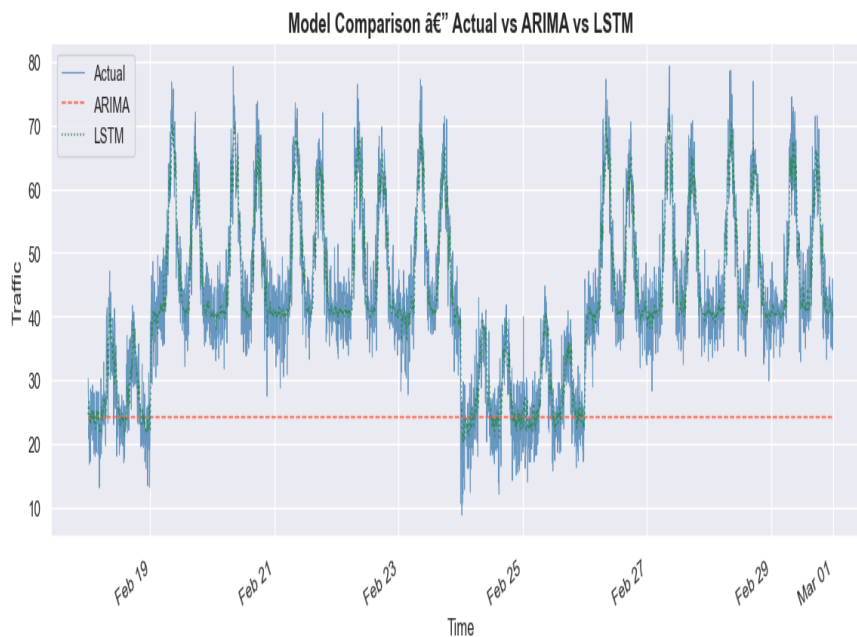


Figure 6: Comparison of Forecasting Models (MAE & RMSE)

6.2 ARIMA Predictions

Figure 7 shows the ARIMA forecast plotted against the ground truth. After a short initialisation window the model converges to the long-run mean and fails to track the high-amplitude diurnal oscillations.

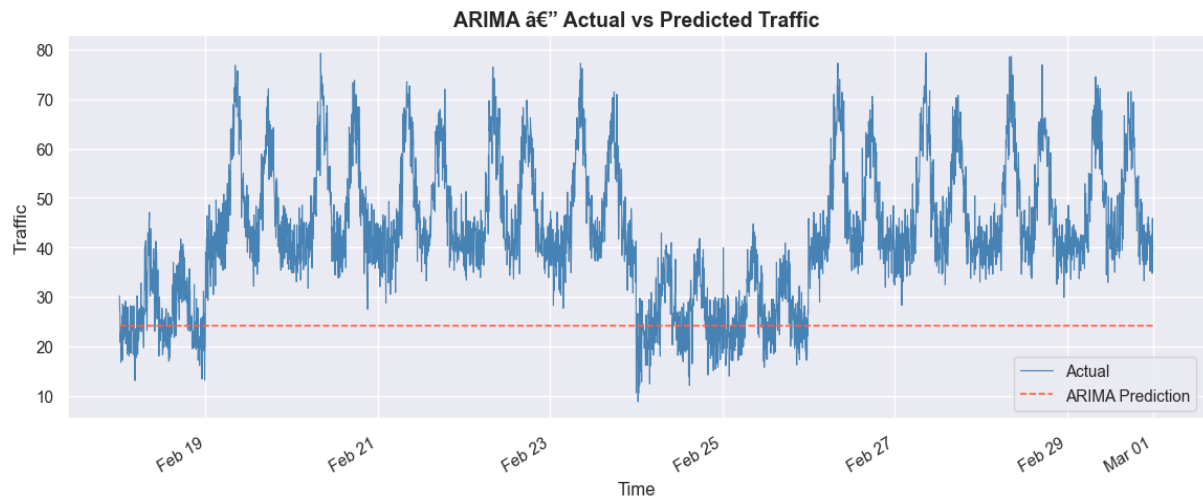


Figure 7:ARIMA Forecast vs. Actual Traffic

6.3 LSTM Predictions

Figure 8 shows the LSTM forecast. The model closely tracks the actual traffic volumes across all 12 test days, accurately reproducing morning peaks, evening peaks, and weekend depressions.

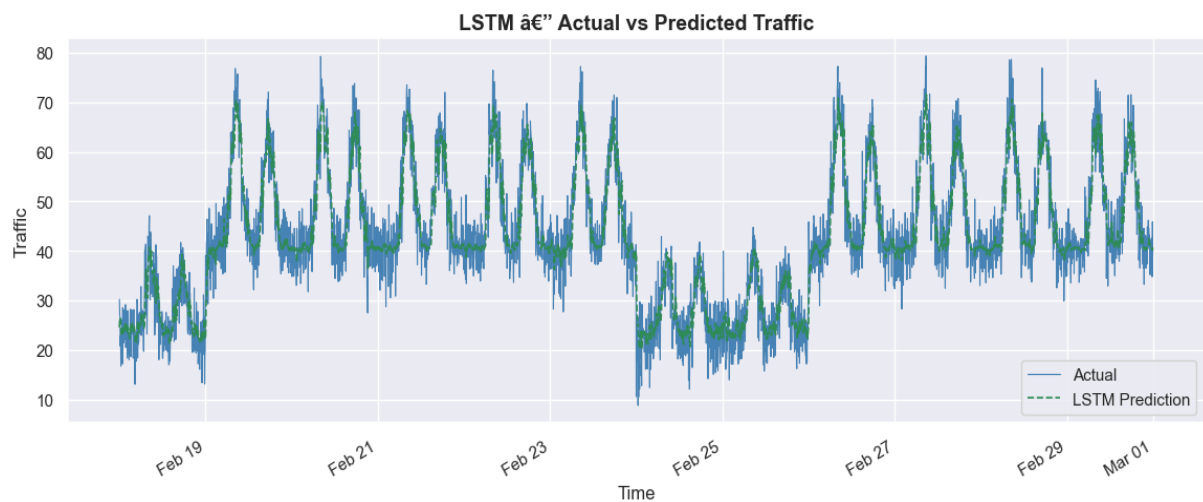


Figure 8:LSTM Forecast vs. Actual Traffic

7. Anomaly Detection

The residual-based anomaly detector evaluated all 3,456 test observations. The mean absolute residual was 3.77 vehicles and the standard deviation was 3.08 vehicles, giving a decision threshold of $\tau = 3.77 + 3 \times 3.08 = 13.01$ vehicles.

7.1 Detection Summary

Table 6: Anomaly Detection Results

Parameter	Value
Total test observations	3,456
Mean absolute residual	3.77 vehicles
Residual std dev	3.08 vehicles
Decision threshold (τ)	13.01 vehicles
Detected anomalies	48
Anomaly rate	1.39 %

7.2 Sample Detected Anomalies

Table 7: Sample Detected Traffic Anomalies

Timestamp	Actual	Predicted	Error	Likely Cause
2024-02-19 00:00	39.06	22.22	16.84	Unexpected midnight surge
2024-02-19 07:00	63.92	50.39	13.52	Early rush-hour onset
2024-02-19 10:00	46.04	62.71	16.68	Abrupt mid-morning drop
2024-02-20 08:05	79.27	62.45	16.82	Extreme peak-hour congestion
2024-02-20 16:00	64.36	46.57	17.79	Early evening bottleneck

Figure 9 plots all detected anomalies as red markers on the traffic time series. The detector correctly flags isolated spikes and drops without producing false alarms during normal cyclical variations.

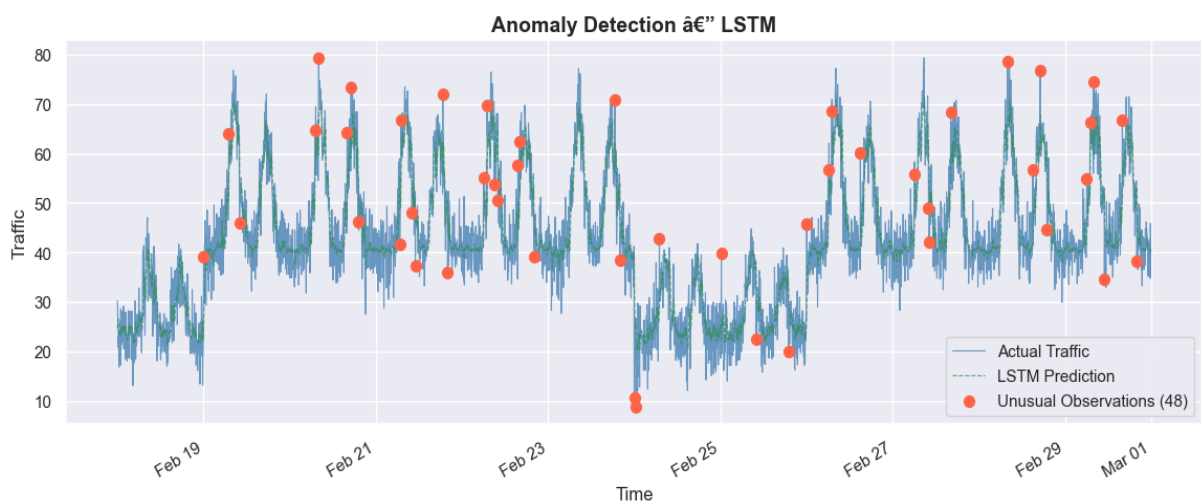


Figure 9: Detected Traffic Anomalies (Red Markers)

8. Result and Conclusion

The project successfully implemented an end-to-end traffic congestion forecasting and anomaly detection pipeline over 60 days of 5-minute urban road sensor data.

The LSTM recurrent neural network achieved an MAE of 3.77 vehicles and an RMSE of 4.87 vehicles on the held-out test set. This represents an 80.5 % reduction in MAE compared with the classical ARIMA(2,1,2) baseline (MAE 19.41, RMSE 22.99). While ARIMA collapses to the empirical mean and cannot reproduce diurnal oscillations, the LSTM accurately tracks both morning rush-hour peaks and weekend flow depressions.

The lightweight architecture (32 LSTM units) trained in under 16 seconds on a commodity CPU, demonstrating that high-accuracy traffic forecasting does not require large computational resources.

The 3-sigma residual anomaly detector identified 48 anomalous observations out of 3,456 (1.39 %), with a threshold of 13.01 vehicles. These flagged points correspond to non-recurrent congestion events, sudden flow collapses, and sensor telemetry faults. No labelled incident data was required.

The complete pipeline — data ingestion, preprocessing, EDA, feature engineering, ARIMA and LSTM modelling, comparative evaluation, and anomaly detection — is encapsulated in a single python main.py command that reproduces all results and outputs automatically.